LA MINUTE DE CODE



FORMATION: JAVASCRIPT DE A À Z

PARTIE 3

LA MINUTE DE CODE

PLAN DE LA FORMATION

JS

- Les méthodes liées aux chaînes de caractères
- Le regex
- La méthode replace
- Les objets
- Les valeurs primitives et les valeurs de références
- Le mot clé This
- La méthode bind
- Les opérateurs ternaires
- L'objet Math
- SetTimeout et SetInterval

LA MINUTE DE CODE

MÉTHODES STRING

JS

La méthode charAt

charAt() : cette méthode renvoie le caractère à l'index spécifié dans une chaîne de caractères. Si l'index est en dehors de la plage de la chaîne de caractères, elle renvoie une chaîne vide. Voici un exemple :

```
let str = "hello";
console.log(str.charAt(1)); // Output: "e"
```

La méthode indexOf

indexOf() : cette méthode renvoie l'index de la première occurrence d'un caractère ou d'une sous-chaîne spécifié dans une chaîne de caractères. Si le caractère ou la sous-chaîne n'est pas trouvé, elle renvoie -1. Voici un exemple :

```
let str = "hello";
console.log(str.indexOf("l")); // Output: 2
```

La méthode charCodeAt

charCodeAt() : cette méthode renvoie le code Unicode du caractère à l'index spécifié dans une chaîne de caractères. Si l'index est en dehors de la plage de la chaîne de caractères, elle renvoie NaN. Voici un exemple :

```
let str = "hello";
console.log(str.charCodeAt(1)); // Output:
101 (le code Unicode pour le caractère "e")
```

LA MINUTE DE CODE

MÉTHODES STRING

JS

La méthode slice

slice() : cette méthode renvoie une sous-chaîne d'une chaîne de caractères, en extrayant une partie de la chaîne de caractères. Elle prend deux arguments : le premier argument est l'index de départ (inclus), et le deuxième argument est l'index de fin (exclu) de la sous-chaîne. Si le deuxième argument est omis, la méthode renvoie la sous-chaîne jusqu'à la fin de la chaîne de caractères. Voici un exemple :

```
let str = "hello";
console.log(str.slice(1, 3)); // Output:
"el"
```

La méthode split

split() : cette méthode divise une chaîne de caractères en un tableau de sous-chaînes, en utilisant un séparateur spécifié. Si le séparateur est omis, la méthode divise la chaîne de caractères en un tableau de caractères individuels. Voici un exemple :

```
let str = "hello,world";
console.log(str.split(",")); // Output:
["hello", "world"]
```

La méthode concat

concat() : cette méthode concatène deux ou plusieurs chaînes de caractères en une seule chaîne de caractères. Voici un exemple :

```
let str1 = "hello";
let str2 = "world";
console.log(str1.concat(" ", str2)); //
Output: "hello world"
```

MÉTHODES STRING

La méthode trim

trim() : cette méthode supprime les espaces blancs au début et à la fin d'une chaîne de caractères. Voici un exemple :

```
let str = "   hello   ";
console.log(str.trim()); // Output: "hello"
```

La méthode includes

La méthode includes() est une méthode liée aux chaînes de caractères en JavaScript qui permet de vérifier si une chaîne de caractères contient une sous-chaîne spécifiée. Elle renvoie une valeur booléenne true si la sous-chaîne est trouvée dans la chaîne de caractères, et false sinon.

```
let str = "hello world";
console.log(str.includes("world")); //
Output: true
console.log(str.includes("foo")); //
Output: false
```

Dans cet exemple, la première instruction renvoie true car la sous-chaîne "world" est trouvée dans la chaîne de caractères "hello world", tandis que la deuxième instruction renvoie false car la sous-chaîne "foo" n'est pas trouvée dans la chaîne de caractères "hello world".

Il est important de noter que la méthode includes() est sensible à la casse, c'est-à-dire qu'elle considère les différences entre les majuscules et les minuscules. Si vous voulez effectuer une recherche sans tenir compte de la casse, vous pouvez convertir la chaîne de caractères et la sous-chaîne en majuscules ou en minuscules avant d'utiliser la méthode includes().

LA MINUTE DE CODE

REGEX

JS

Les expressions régulières (regex) sont des motifs de texte qui permettent de rechercher et de manipuler des chaînes de caractères selon des règles spécifiques.

Les regex sont utilisées pour effectuer des opérations telles que la validation de formulaire, la recherche de correspondances de texte, la substitution de texte, etc.

Il existe différents paramètres et options que l'on peut utiliser avec les regex pour personnaliser leur comportement. Voici quelques-uns des paramètres les plus couramment utilisés :

- i : Ce paramètre permet d'ignorer la casse des caractères. Par exemple, si vous utilisez la regex /chat/i, elle correspondra à "chat", "Chat", "CHAT", etc.
- g : Ce paramètre permet d'effectuer une recherche globale dans une chaîne de caractères. Cela signifie qu'il va rechercher toutes les occurrences de la regex dans la chaîne de caractères plutôt que de s'arrêter après avoir trouvé la première occurrence.
- m : Ce paramètre permet de rechercher des correspondances multilignes dans une chaîne de caractères. Cela signifie que la regex peut correspondre à des expressions qui s'étendent sur plusieurs lignes.
- s : Ce paramètre permet de traiter la chaîne de caractères comme si elle était une seule ligne. Cela signifie que les caractères de retour à la ligne et les espaces blancs ne sont pas pris en compte dans la correspondance.
- x : Ce paramètre permet d'ignorer les espaces et les commentaires dans la regex, ce qui la rend plus lisible et plus facile à comprendre.

Il existe également d'autres paramètres et options que l'on peut utiliser avec les regex, tels que le mode "insensible aux accents" pour prendre en compte les caractères accentués, le mode "unicode" pour prendre en compte les caractères Unicode, etc.

Il est important de bien comprendre les paramètres et options que l'on utilise avec les regex pour obtenir des résultats précis et cohérents.

pour matcher des caractères spécifiques à l'aide de regex, on peut les mettre entre crochets []. Voici quelques exemples de caractères que l'on peut matcher avec des crochets en regex :

- Les lettres minuscules : [a-z]
- Les lettres majuscules : [A-Z]
- Les chiffres : [0-9]
- Les lettres et les chiffres : [a-zA-Z0-9]
- Les caractères spéciaux : [!@#$%^&*()]

Par exemple, si l'on veut matcher une chaîne de caractères qui commence par une lettre majuscule, suivie de plusieurs lettres minuscules, on peut utiliser la regex suivante : /^[A-Z][a-z]+/.

Cette regex va correspondre à des chaînes de caractères comme "Bonjour", "Hello", "Salut", etc.

De même, si l'on veut matcher une chaîne de caractères qui contient uniquement des lettres et des chiffres, on peut utiliser la regex suivante : /^[a-zA-Z0-9]+/. Cette regex va correspondre à des chaînes de caractères comme "MotDePasse123", "MyPassword456", "CodeSecret789", etc.

Il est important de noter que les crochets ne peuvent matcher qu'un seul caractère à la fois. Par exemple, si l'on veut matcher une chaîne de caractères qui commence par "http" ou "https", on ne peut pas utiliser la regex [httphttps], car cela va matcher soit "h", soit "t", soit "p", soit "s".

Dans ce cas, on peut utiliser la regex suivante : /^https?/. Cette regex va correspondre à des chaînes de caractères comme "http://example.com", "https://example.com", "http://www.example.com", etc.

Exemple pour vérifier une adresse mail:

```
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
const email = "example@example.com";

if(emailRegex.test(email)) {
  console.log("L'adresse email est valide.");
} else {
  console.log("L'adresse email n'est pas valide.");
}
```

Cette regex permet de vérifier si une adresse email est valide en s'assurant qu'elle contient un seul "@" et un seul "." après le "@". Elle va correspondre à des adresses email comme "example@example.com", "john.doe@example.co.uk", "jane.doe@example.com", etc.

REGEX

Exemple pour une recherche de correspondances dans une chaîne de caractères :

```
const text = "Le chat noir est sur le toit";
const regex = /chat/i;

if(regex.test(text)) {
  console.log("Le mot 'chat' a été trouvé dans le texte.");
} else {
  console.log("Le mot 'chat' n'a pas été trouvé dans le texte.");
}
```

Cette regex permet de rechercher une correspondance de texte dans une chaîne de caractères en cherchant le mot "chat", sans tenir compte de la casse. Elle va correspondre à des chaînes de caractères comme "Le chat noir est sur le toit", "J'ai un Chat", "Les chats sont mignons", etc.

LA MINUTE DE CODE

REPLACE

JS

La méthode replace() est une méthode de chaîne de caractères en JavaScript qui permet de remplacer une sous-chaîne spécifiée dans une chaîne de caractères par une autre sous-chaîne. Cette méthode prend deux arguments : le premier argument est la sous-chaîne à remplacer, et le deuxième argument est la sous-chaîne qui sera utilisée pour remplacer la première.

La méthode replace() peut également utiliser des expressions régulières pour effectuer des remplacements plus complexes. Par exemple, la chaîne de caractères "hello world" peut être remplacée par "goodbye world" en utilisant l'expression régulière /hello/ et la sous-chaîne "goodbye" comme suit :

```
let str = "hello world";
let newStr = str.replace(/hello/, "goodbye");
console.log(newStr); // Output: "goodbye world"
```

Dans cet exemple, la méthode replace() recherche la sous-chaîne "hello" dans la chaîne de caractères "hello world" en utilisant l'expression régulière /hello/, puis la remplace par la sous-chaîne "goodbye" pour obtenir la nouvelle chaîne de caractères "goodbye world".

LA MINUTE DE CODE

LES OBJETS

JS

En JavaScript, tout est un objet. Les objets en JavaScript sont des entités complexes qui peuvent avoir des propriétés et des méthodes. Les objets sont créés à partir de classes, qui définissent les propriétés et les méthodes de l'objet.

Cependant, contrairement à d'autres langages orientés objet, JavaScript utilise un modèle d'objet dynamique, ce qui signifie que les propriétés et les méthodes d'un objet peuvent être ajoutées, modifiées ou supprimées à tout moment.

Les objets en JavaScript peuvent être de différents types, tels que les objets natifs, les objets hérités, les objets créés par l'utilisateur, etc. Les objets natifs comprennent des types de données tels que les chaînes de caractères, les nombres, les tableaux, les fonctions, les dates, les expressions régulières, etc.

Les objets hérités sont des objets qui héritent de certaines propriétés et méthodes d'autres objets. Les objets créés par l'utilisateur sont des objets créés par le développeur pour répondre à des besoins spécifiques.

Les propriétés d'un objet en JavaScript sont des paires clé-valeur qui stockent des données. Les propriétés peuvent être des valeurs primitives telles que des chaînes de caractères, des nombres, des booléens, etc., ou des objets eux-mêmes. Les méthodes d'un objet en JavaScript sont des fonctions qui peuvent être appelées sur l'objet pour effectuer des actions ou renvoyer des valeurs.

Les objets en JavaScript sont très flexibles et puissants, mais ils peuvent être complexes à utiliser et à manipuler. Il est important de comprendre les différents types d'objets et les méthodes et propriétés qui leur sont associées pour pouvoir les utiliser efficacement dans votre code.

LA MINUTE DE CODE

LES OBJETS

JS

En JavaScript, un objet est donc une structure de données complexe qui peut contenir des propriétés et des méthodes.

Les propriétés d'un objet sont des paires clé-valeur, où chaque clé est une chaîne de caractères qui identifie la propriété, et chaque valeur peut être de n'importe quel type de données JavaScript, y compris des fonctions, des nombres, des chaînes de caractères, des booléens, des tableaux et même d'autres objets.

Voici comment accéder à une propriété d'un objet en JavaScript :

```
const person = {
  name: "John",
  age: 30,
  isMarried: true,
  hobbies: ["reading", "swimming",
"traveling"]
};

console.log(person.name); // "John"
console.log(person.age); // 30
console.log(person.isMarried); // true
console.log(person.hobbies); // ["reading",
"swimming", "traveling"]
```

Dans cet exemple, nous avons créé un objet person avec plusieurs propriétés. Nous pouvons accéder à chaque propriété en utilisant la notation point, en écrivant le nom de l'objet suivi d'un point et du nom de la propriété.

Voici comment ajouter une propriété à un objet en JavaScript :

```
person.nationality = "French";
console.log(person.nationality); // "French"
```

Dans cet exemple, nous avons ajouté une nouvelle propriété nationality à l'objet person. Nous pouvons maintenant accéder à cette propriété en utilisant la notation point comme nous l'avons vu précédemment.

LA MINUTE DE CODE

PRIMITIF ET RÉFÉRENCES

JS

En JavaScript, il existe deux types de valeurs : les valeurs primitives et les valeurs de référence.

Les valeurs primitives sont des données simples stockées directement dans la variable. Il existe six types de valeurs primitives en JavaScript :

- Nombre (ex: 10)
- Chaîne de caractères (ex: "hello world")
- Booléen (ex: true ou false)
- Null (ex: null)
- Undefined (ex: undefined)
- Symbole (ex: Symbol("foo"))

Les valeurs primitives sont copiées par valeur, ce qui signifie qu'une nouvelle copie de la valeur est créée lorsque la variable est affectée à une autre variable ou passée en argument à une fonction.

Les valeurs de référence, en revanche, sont des objets qui sont stockés en mémoire et référencés par une variable. Les valeurs de référence comprennent les objets, les tableaux, les fonctions et les autres types d'objets.

Lorsque vous créez une variable qui contient une valeur de référence, la variable ne stocke pas réellement la valeur de l'objet. Au lieu de cela, il stocke une référence à l'emplacement en mémoire où l'objet est stocké.

LA MINUTE DE CODE

DELETE

Pour supprimer une propriété d'un objet en JavaScript, vous pouvez utiliser l'opérateur delete.

Voici un exemple montrant comment supprimer une propriété d'un objet

```
const calculator = {
  add: function(a, b) {
    return a + b;
  },
  subtract: function(a, b) {
    return a - b;
  }
};

console.log(calculator.add(2, 3)); // 5
console.log(calculator.subtract(5, 3)); // 2
```

Dans cet exemple, nous avons supprimé la propriété isMarried de l'objet person en utilisant l'opérateur delete. Nous pouvons maintenant voir que la propriété a été supprimée en affichant l'objet person dans la console.

Notez que si vous essayez d'accéder à une propriété qui n'existe pas dans l'objet, JavaScript renverra simplement undefined plutôt qu'une erreur.

LA MINUTE DE CODE

PRIMITIF ET RÉFÉRENCES

Les valeurs de référence sont copiées par référence, ce qui signifie que lorsque vous affectez une variable qui contient une valeur de référence à une autre variable, les deux variables référencent le même objet en mémoire.

Voici un exemple montrant la différence entre les valeurs primitives et les valeurs de référence en JavaScript :
javascript

```javascript
let x = 10;
let y = x; // La valeur 10 est copiée dans la variable y

x = 20;

console.log(x); // 20
console.log(y); // 10

let obj1 = { name: "John" };
let obj2 = obj1; // La variable obj2 référence le même objet que la variable obj1

obj1.name = "Mike";

console.log(obj1.name); // "Mike"
console.log(obj2.name); // "Mike"
```

PRIMITIF ET RÉFÉRENCES

Dans cet exemple, lorsque nous avons affecté la variable x à la variable y, une nouvelle copie de la valeur primitive a été créée.

Cependant, lorsque nous avons affecté la variable obj1 à la variable obj2, les deux variables ont référencé le même objet en mémoire.

Par conséquent, lorsque nous avons modifié la propriété name de l'objet en utilisant la variable obj1, le changement a également été reflété dans la variable obj2.

En résumé, les valeurs primitives sont des données simples stockées directement dans la variable et copiées par valeur, tandis que les valeurs de référence sont des objets stockés en mémoire et référencés par une variable, et sont copiées par référence.

LA MINUTE DE CODE

THIS

JS

En JavaScript, le mot-clé "this" fait référence à l'objet courant sur lequel une méthode est appelée ou à l'objet global (window) si la méthode est appelée directement dans le contexte global.

La valeur de "this" dépend donc du contexte dans lequel il est utilisé. Si "this" est utilisé dans une fonction, la valeur de "this" dépendra de la façon dont la fonction est appelée. Si la fonction est appelée en tant que méthode d'un objet, alors "this" fera référence à cet objet.

Si la fonction est appelée directement dans le contexte global, alors "this" fera référence à l'objet global (window).

Voici un exemple:

```
const myObject = {
  myMethod: function() {
    console.log(this); // fera référence à myObject
  }
};

myObject.myMethod(); // appel de la méthode sur l'objet

function myFunction() {
  console.log(this); // fera référence à l'objet global
(window)
}

myFunction(); // appel de la fonction directement dans
le contexte global
```

LA MINUTE DE CODE

LA MÉTHODE BIND

JS

La méthode "bind" en JavaScript permet de lier une fonction à un objet spécifique, en fixant le contexte "this" de la fonction à l'objet spécifié. La méthode "bind" renvoie une nouvelle fonction qui peut être appelée avec le contexte "this" fixé.

Voici un exemple pour mieux comprendre :

```
const person = {
  name: "John",
  age: 30,
  greet: function() {
    console.log("Hello, my name is " + this.name + "
and I am " + this.age + " years old.");
  }
};

const greetJohn = person.greet.bind(person); // ici
nous créons une nouvelle fonction 'greetJohn' en liant
le contexte 'this' de la fonction 'greet' à l'objet
'person'

greetJohn(); // 'Hello, my name is John and I am 30
years old.'
```

LA MÉTHODE BIND

Dans cet exemple, la méthode "bind" est utilisée pour lier la méthode "greet" à l'objet "person", en fixant le contexte "this" à "person". La variable "greetJohn" contient la nouvelle fonction liée, qui peut être appelée avec le contexte "this" fixé.

La méthode "bind" est utile dans les situations où nous voulons utiliser une méthode d'un objet spécifique dans un contexte différent, par exemple lors de l'utilisation de callbacks ou de setTimeout. En liant le contexte "this" de la méthode à l'objet spécifique, nous pouvons nous assurer que la méthode est appelée avec les bonnes valeurs de propriétés et de variables.

LA MINUTE DE CODE

OPÉRATEURS TERNAIRES

JS

En JavaScript, l'opérateur ternaire (aussi appelé "opérateur conditionnel") est un opérateur qui permet d'évaluer une expression booléenne et de renvoyer une valeur en fonction de cette évaluation.

L'opérateur ternaire a la syntaxe suivante :

```
condition ? valeurSiVrai : valeurSiFaux
```

Si la condition est vraie, l'opérateur ternaire renvoie la valeur située avant le signe "?", sinon il renvoie la valeur située après le signe ":".
Voici un exemple pour mieux comprendre :

```
const age = 25;

const message = age >= 18 ? "Vous êtes
majeur." : "Vous êtes mineur.";

console.log(message); // "Vous êtes
majeur."
```

OPÉRATEURS TERNAIRES

Dans cet exemple, l'opérateur ternaire est utilisé pour évaluer la condition "age >= 18". Si cette condition est vraie, l'opérateur ternaire renvoie la chaîne de caractères "Vous êtes majeur.", sinon il renvoie la chaîne de caractères "Vous êtes mineur.". Dans ce cas, la condition est vraie (25 est supérieur ou égal à 18), donc la variable "message" contient la chaîne de caractères "Vous êtes majeur.".

L'opérateur ternaire est utile pour simplifier le code et rendre les expressions conditionnelles plus courtes et plus lisibles. Cependant, il peut parfois rendre le code moins lisible s'il est utilisé de manière excessive ou complexe. Il est donc important de l'utiliser avec parcimonie et de privilégier la lisibilité du code.

LA MINUTE DE CODE

L'OBJET MATH

JS

En JavaScript, l'objet "Math" est un objet intégré qui contient des propriétés et des méthodes pour effectuer des opérations mathématiques.

Cet objet ne peut pas être instancié, car il s'agit d'un objet statique.

Voici quelques-unes des propriétés et méthodes disponibles dans l'objet "Math" :

- "Math.PI" : une propriété qui contient la valeur de Pi (3,14159...).
- "Math.abs(x)" : une méthode qui renvoie la valeur absolue d'un nombre.
- "Math.ceil(x)" : une méthode qui renvoie le plus petit entier supérieur ou égal à un nombre donné.
- "Math.floor(x)" : une méthode qui renvoie le plus grand entier inférieur ou égal à un nombre donné.
- "Math.max(x, y, ...)" : une méthode qui renvoie le plus grand nombre d'une série de nombres.
- "Math.min(x, y, ...)" : une méthode qui renvoie le plus petit nombre d'une série de nombres.
- "Math.pow(x, y)" : une méthode qui renvoie la valeur de x élevée à la puissance y.
- "Math.round(x)" : une méthode qui renvoie la valeur d'un nombre arrondi à l'entier le plus proche.
- "Math.sqrt(x)" : une méthode qui renvoie la racine carrée d'un nombre.

Voici un exemple pour mieux comprendre :

LA MINUTE DE CODE

L'OBJET MATH

JS

```
const x = 5.5;
const y = 2;

const absX = Math.abs(x); // 5.5
const ceilX = Math.ceil(x); // 6
const floorX = Math.floor(x); // 5
const maxXY = Math.max(x, y); // 5.5
const powXY = Math.pow(x, y); // 30.25
const roundX = Math.round(x); // 6
const sqrtX = Math.sqrt(x); // 2.345207879911715
```

Dans cet exemple, nous utilisons certaines des propriétés et méthodes disponibles dans l'objet "Math" pour effectuer des opérations mathématiques sur les nombres "x" et "y".

SETINTERVAL ET SETTIMEOUT

En JavaScript, les fonctions "setTimeout" et "setInterval" sont des fonctions de temporisation qui permettent de programmer l'exécution d'une fonction après un certain délai.

La fonction "setTimeout" permet de programmer l'exécution d'une fonction une seule fois après un délai donné en millisecondes.

Elle a la syntaxe suivante :

```
setTimeout(function, delay);
```

où "function" est la fonction à exécuter et "delay" est le délai en millisecondes avant l'exécution de la fonction.
Voici un exemple pour mieux comprendre :

```
function sayHello() {
  console.log("Hello !");
}

setTimeout(sayHello, 3000); // affiche "Hello !" dans la console après 3 secondes
```

Dans cet exemple, la fonction "sayHello" est programmée pour être exécutée une seule fois après un délai de 3 secondes.

SETINTERVAL ET SETTIMEOUT

La fonction "setInterval" permet de programmer l'exécution répétée d'une fonction toutes les "x" millisecondes. Elle a la syntaxe suivante :

```
setInterval(function, delay);
```

où "function" est la fonction à exécuter et "delay" est le délai en millisecondes entre chaque exécution de la fonction.
Voici un exemple pour mieux comprendre :

```
let counter = 0;

function incrementCounter() {
  counter++;
  console.log(counter);
}

setInterval(incrementCounter, 1000); // affiche un compteur qui incrémente toutes les secondes
```

Dans cet exemple, la fonction "incrementCounter" est programmée pour être exécutée toutes les secondes grâce à la fonction "setInterval". Elle incrémente un compteur à chaque exécution et affiche la valeur du compteur dans la console.

Il est important de noter que les fonctions "setTimeout" et "setInterval" peuvent être annulées à tout moment en utilisant la fonction "clearTimeout" ou "clearInterval" respectivement. Cela permet d'annuler une temporisation en cours et d'empêcher l'exécution de la fonction programmée.



laminutedecode.io

@LAMINUTEDECODE

